

Enterprise Prompt: Build a Production-Grade MCP Server for Microsoft SQL Server using .NET

You are a Principal Software Architect, Staff Software Engineer, Database Platform Engineer, Security Engineer, and DevOps Engineer with deep expertise in:

- MCP (Model Context Protocol)
- .NET 8 / .NET 9
- C#
- Microsoft SQL Server
- Entity Framework Core
- ADO.NET
- Dapper
- Enterprise Security
- Distributed Systems
- Observability
- Production Architecture
- Claude Desktop MCP Integration
- AI Tooling Infrastructure

Your task is to design and implement a complete enterprise-grade MCP Server that exposes Microsoft SQL Server capabilities to Claude through MCP.

The implementation must be production-ready, secure, scalable, maintainable, and suitable for large enterprise environments.

Objective

Build a fully functional MCP Server in .NET that allows Claude to:

- Discover SQL Server metadata
- Execute SQL queries safely
- Explore database schema
- Inspect tables
- Inspect views
- Inspect stored procedures
- Inspect scalar functions
- Inspect table-valued functions (TVF)
- Inspect triggers

- Inspect user-defined table types
- Inspect user-defined data types
- Inspect indexes
- Inspect foreign keys
- Inspect constraints
- Retrieve execution plans
- Analyze dependencies
- Generate documentation
- Perform schema comparison
- Execute administrative operations (optional and role-based)

The MCP server must expose all capabilities through MCP tools, resources, and prompts.

Architecture Requirements

Design a layered architecture:

Layers

Presentation Layer

MCP Server

Application Layer

Business Services

Domain Layer

Entities

Models

Contracts

Infrastructure Layer

SQL Server Access

Repositories

Caching

Telemetry

Cross-Cutting Layer

Logging

Security

Validation

Auditing

Metrics

Technology Stack

Use:

- .NET 8
- C#
- ASP.NET Core
- Model Context Protocol SDK
- Dapper
- ADO.NET
- Microsoft.Data.SqlClient
- FluentValidation
- Serilog
- OpenTelemetry
- Polly
- Memory Cache
- Distributed Cache Support
- xUnit
- Moq
- TestContainers
- Docker

MCP Features

Implement all MCP primitives.

Tools

Create MCP tools for:

Database Discovery

get_databases

get_schemas

get_tables

get_views

get_functions

get_procedures
get_triggers
get_user_defined_types
get_table_types
get_indexes
get_constraints
get_foreign_keys
get_sequences
get_synonyms

Table Operations

describe_table
get_table_columns
get_primary_keys
get_table_relationships
get_table_statistics
get_row_count
sample_table_data
search_table_data

View Operations

describe_view
get_view_definition
get_view_dependencies
get_view_columns

Function Operations

describe_function
get_function_definition
get_scalar_functions
get_table_valued_functions

analyze_function_dependencies

Stored Procedure Operations

describe_procedure

get_procedure_definition

get_procedure_parameters

execute_procedure

analyze_procedure_dependencies

Trigger Operations

describe_trigger

get_trigger_definition

get_trigger_dependencies

Type Operations

describe_user_defined_type

describe_table_type

get_type_definition

Query Operations

execute_query

execute_parameterized_query

validate_query

estimate_query_cost

get_execution_plan

analyze_query

format_query

Dependency Analysis

find_object_dependencies

find_referencing_objects

generate_dependency_graph

generate_erd

Documentation

generate_database_documentation

generate_schema_documentation

generate_table_documentation

generate_api_documentation

Schema Comparison

compare_schemas

compare_databases

generate_migration_script

Health

health_check

database_connectivity_test

cache_status

telemetry_status

SQL Server Metadata Coverage

Support:

INFORMATION_SCHEMA

sys.tables

sys.views

sys.objects

sys.columns

sys.types

sys.schemas

sys.parameters

sys.procedures

sys.sql_modules
sys.triggers
sys.indexes
sys.foreign_keys
sys.key_constraints
sys.check_constraints
sys.default_constraints
sys.table_types
sys.synonyms
sys.sequences
sys.dm_exec_query_stats
sys.dm_exec_sql_text
sys.dm_exec_query_plan
sys.sql_expression_dependencies

Security Requirements

Implement enterprise-grade security.

Authentication

Support:

- Windows Authentication
 - SQL Authentication
 - Azure Managed Identity
 - Azure AD Authentication
-

Authorization

Implement:

Role-based access control (RBAC)

Roles:

- Admin
- DBA
- Developer

- ReadOnly
 - Auditor
-

Query Safety

Prevent:

- DROP DATABASE
- DROP TABLE
- ALTER LOGIN
- SHUTDOWN
- xp_cmdshell
- dynamic OS commands
- linked server abuse

Implement:

- SQL allowlist
 - SQL denylist
 - query validation
 - query timeout
 - execution limits
-

Secrets

Use:

- Azure Key Vault
- Environment Variables
- Secret Manager

Never hardcode secrets.

Performance Requirements

Implement:

Connection Pooling

Async Operations

Streaming Results

Pagination

Batch Processing

Caching

Compression

Circuit Breakers

Retry Policies

Request Throttling

Query Cost Limits

Large Result Set Protection

Observability

Implement:

Logging

Serilog

Structured Logs

Correlation IDs

Request IDs

Audit Logs

Metrics

OpenTelemetry

Prometheus-compatible metrics

Metrics for:

- Query duration
 - Tool execution duration
 - Cache hit rate
 - Error rate
 - Active connections
-

Tracing

Distributed tracing

Activity propagation

MCP request tracing

Database tracing

Configuration

Use strongly typed configuration.

Example:

McpSettings

SqlServerSettings

SecuritySettings

TelemetrySettings

CacheSettings

ToolSettings

Error Handling

Create:

Domain Exceptions

Validation Exceptions

Database Exceptions

Security Exceptions

Tool Exceptions

Transient Exceptions

Return structured MCP errors.

AI-Oriented Features

Claude should be able to ask:

"Show all tables in schema Sales"

"Describe Customer table"

"Find all stored procedures referencing Customer"

"Generate documentation for Sales schema"

"List all TVFs"

"Show trigger definitions"

"Find all foreign keys"

"Generate ERD"

"Analyze this query"

"Show execution plan"

"Compare Production and Staging schemas"

"Generate migration script"

"Find unused objects"

"Find missing indexes"

"Explain database structure"

The MCP server should return structured, AI-friendly responses.

Resources

Create MCP resources for:

database://schemas

database://tables

database://views

database://functions

database://procedures

database://triggers

database://types

database://indexes

database://relationships

database://documentation

database://dependency-graph

Prompts

Create reusable MCP prompts:

database-analysis

query-analysis

schema-documentation

erd-generation
dependency-analysis
migration-generation
performance-review
security-review

Claude Desktop Integration

Provide:

1. MCP server startup configuration
 2. Claude Desktop configuration example
 3. Windows setup
 4. Linux setup
 5. Docker setup
 6. Docker Compose setup
 7. Kubernetes deployment
 8. Azure deployment
-

Testing Requirements

Implement:

Unit Tests

Integration Tests

Database Tests

Contract Tests

Load Tests

Security Tests

MCP Tool Tests

Target:

- 90% code coverage
 - Full CI/CD support
-

CI/CD

Provide:

GitHub Actions

Azure DevOps Pipeline

Build

Test

Security Scan

Container Build

Deployment

Versioning

Release Notes

Deliverables

Generate:

1. Complete solution architecture
2. Folder structure
3. Domain model
4. Database metadata model
5. MCP tool implementations
6. Service layer
7. Repository layer
8. Security layer
9. Telemetry layer
10. Configuration layer
11. Docker files
12. Kubernetes manifests
13. Claude Desktop configuration
14. Complete source code
15. Unit tests
16. Integration tests
17. README
18. Deployment guide

19. Operations guide

20. Troubleshooting guide

Output production-quality code only.

Use SOLID principles, Clean Architecture, Domain-Driven Design, and enterprise best practices throughout.

The generated solution must be deployable immediately in a real enterprise environment.